

#15 Swagger/OpenAPI Documentation

Swagger/OpenAPI Documentation	API documentation, endpoint testing, Swagger UI configuration
-------------------------------	---

API Documentation

API Documentation is a document that explains:

- What APIs are available
- What request data should be sent
- What response data will be returned
- Authentication requirements
- Error responses
- API examples

Without documentation:

- **Frontend Developer:** "Which API should I call?"
- **Backend Developer:** "Check the code."
- **With Swagger/OpenAPI:**

Frontend Developer:

Open Swagger UI

→ See all APIs

→ Test APIs directly

→ View request & response format

OpenAPI

OpenAPI is a specification (standard format) for describing REST APIs.

It defines:

- Endpoints
- Methods
- Request Parameters

- Request Body
- Responses
- Authentication

Example OpenAPI Description:

POST /students

Request:

```
{  
  "name": "Ummed Singh",  
  "email": "ummed@gmail.com"  
}
```

Response:

```
{  
  "id": 1,  
  "name": "Ummed Singh",  
  "email": "ummed@gmail.com"  
}
```

Swagger

Swagger is a tool that implements the OpenAPI specification.

It provides:

Swagger UI: Interactive webpage where APIs can be tested.

Swagger Documentation: Automatically generated API documentation.

API Testing: No need for Postman for quick testing.

OpenAPI vs Swagger

OpenAPI	Swagger
Specification	Tool
Standard Format	Implementation
Defines API Structure	Generates UI
API Contract	API Visualization

Example

Suppose you're creating a Student Management System.

APIs:

POST /students
GET /students
GET /students/{id}
PUT /students/{id}
DELETE /students/{id}

Swagger automatically generates:

Student Controller

POST Create Student
GET Get All Students
GET Get Student By ID
PUT Update Student
DELETE Delete Student

Spring Boot Swagger Setup

Dependency

For **Spring Boot 3+**

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version>
</dependency>
```

Project Structure

```
src
|
|-- controller
|   StudentController
|
|-- service
|   StudentService
|
|-- entity
|   Student
|
|-- config
|   OpenApiConfig
|
+-- SpringBootApplication
```

Student Entity

```
@Entity
public class Student {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    private String email;

}
```

Student Controller

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @PostMapping
    public Student createStudent(@RequestBody Student student) {
        return student;
    }

    @GetMapping
    public List<Student> getStudents() {

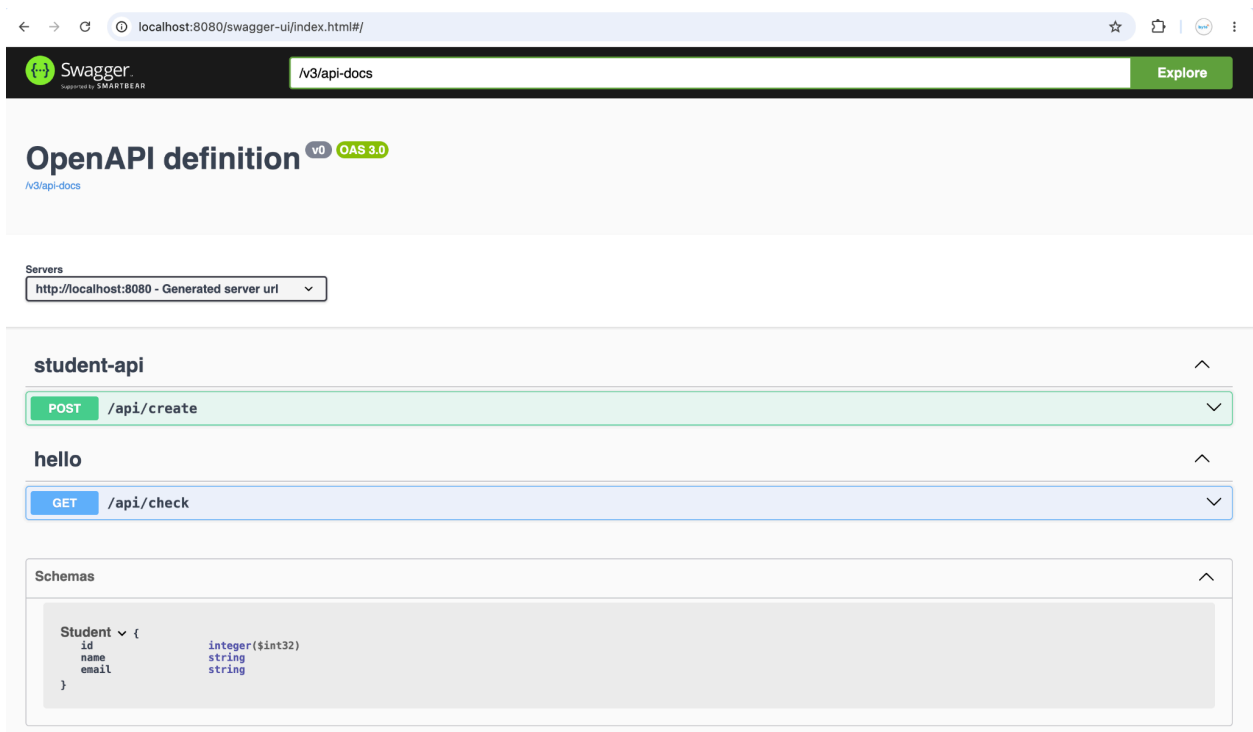
        return List.of(new Student(1L, "Ummed Singh", "ummed@gmail.com"));
    }
}
```

Run Application

After starting application: <http://localhost:8080/swagger-ui.html>

Or: <http://localhost:8080/swagger-ui/index.html>

Swagger UI appears automatically.



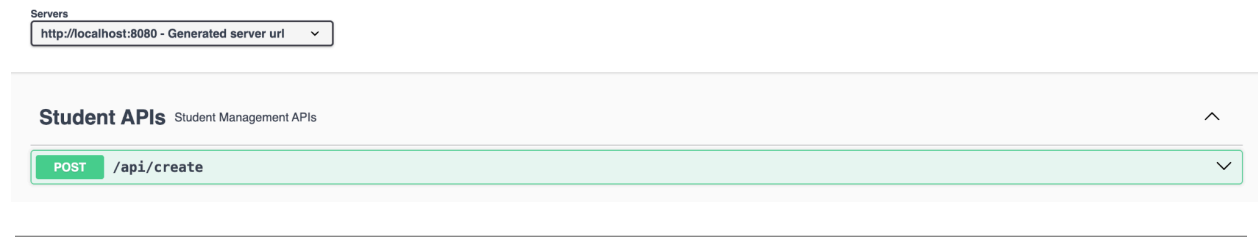
API Documentation Annotations

@Tag

Group APIs.

```
12 | import io.swagger.v3.oas.annotations.tags.Tag;
13 | 
14 | @Tag(name = "Student APIs",
15 |     description = "Student Management APIs")
16 | @RestController
17 | @RequestMapping("/api")
18 | public class StudentApi {
19 | 
20 |
```

Swagger Output:



@Operation

Describe endpoint.

```
@Operation(summary = "Create Student",
            description = "Creates a new student record")
@PostMapping
public Student createStudent(@RequestBody Student student) {
    return student;
}
```

Swagger shows:

Create Student
Creates a new student record

@Parameter

Documents path/query parameters.

```
@GetMapping("/{id}")
public Student getStudent(@Parameter(description = "Student ID")
                          @PathVariable Long id) {

    return service.findById(id);
}
```

@Schema

Documents model fields.

```
public class Student {  
  
    @Schema(description = "Student Identifier", example = "1")  
    private Long id;  
  
    @Schema(description = "Student Name", example = "Ummed Singh")  
    private String name;  
  
    @Schema(description = "Student Email", example =  
"ummed@gmail.com")  
    private String email;  
}
```

Swagger automatically displays examples.

@ApiResponse

Documents responses.

```
@Operation(summary = "Get Student")  
@ApiResponses({@ApiResponse(responseCode = "200", description = "Student Found"),  
                @ApiResponse(responseCode = "404", description = "Student Not Found")})  
@GetMapping("/{id}")  
public Student getStudent(@PathVariable Long id){  
  
    return service.findById(id);  
}
```

Endpoint Testing using Swagger UI

Suppose API:

```
@PostMapping  
public Student createStudent(@RequestBody Student student){  
    return student;  
}
```

Swagger UI shows:

POST /students

Click: Try it out

Request:

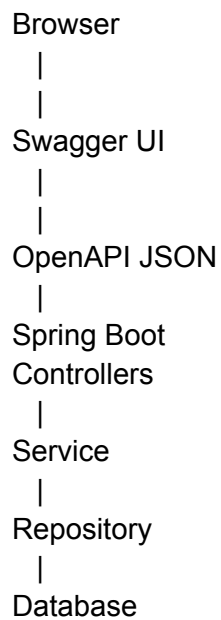
```
{  
  "name": "Ummed Singh",  
  "email": "ummed@gmail.com"  
}
```

Click: Execute

Swagger sends request and displays:

```
{  
  "id": 1,  
  "name": "Ummed Singh",  
  "email": "ummed@gmail.com"  
}
```

Swagger UI Architecture



Flow:

Controller
|
Annotations
|
OpenAPI Generator
|
swagger.json
|
Swagger UI
|
Browser

Custom Swagger Configuration

Create configuration class.

```
@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {

        return new OpenAPI()
            .info(new Info().title("Student Management API")
                .version("1.0")
                .description("Spring Boot Swagger Example")
                .contact(new Contact().name("Ummed Singh")
                    .email("ummed@gmail.com")))
            ;
    }
}
```

Swagger UI Header:

Student Management API
Version 1.0
Spring Boot Swagger Example

Configure Swagger Path

application.properties: springdoc.swagger-ui.path=/api-docs

Access: http://localhost:8080/api-docs

OpenAPI JSON Endpoint

Swagger automatically creates: http://localhost:8080/v3/api-docs

Output:

```
{
  "openapi": "3.0.1",
  "info": {
    "title": "Student API"
  }
}
```

This JSON is the actual OpenAPI specification.

Swagger with JWT Authentication

Without JWT:

Open Swagger
→ Execute API

With JWT:

Open Swagger
→ Authorize
→ Enter Token
→ Execute API

Configuration:

```
@Bean
public OpenAPI customOpenAPI() {

    return new OpenAPI()
        .components(new Components()
            .addSecuritySchemes("BearerAuth",
                new SecurityScheme()
                    .type(SecurityScheme.Type.HTTP)
                    .scheme("bearer")
                    .bearerFormat("JWT")
            ))
        .addSecurityItem(
            new SecurityRequirement()
                .addList("BearerAuth")
        );
}
```

Swagger shows:

Authorize 

Enter: Bearer eyJhbGciOi...

Now secured APIs can be tested.

Project Example

Imagine your Job Portal project.

APIs:

POST /auth/register

POST /auth/login

GET /jobs

POST /jobs

GET /profile

PUT /profile

Swagger Benefits:

Frontend Team:

Can test APIs

QA Team:

Can validate endpoints

Developers:

Can understand APIs

Clients:

Can see available APIs

Interview Questions

What is Swagger?

Swagger is a tool used to generate interactive API documentation based on the OpenAPI specification.

What is OpenAPI?

OpenAPI is a standard specification used to describe REST APIs.

Which dependency is used in Spring Boot 3?

springdoc-openapi-starter-webmvc-ui

Swagger UI URL?

/swagger-ui/index.html

OpenAPI JSON URL?

/v3 / api-docs

Why use Swagger?

Ummed Singh 

MTS@ByteXL | Ex - Nagarro | Java |
Spring Boot | Microservices | React...

- ✓ API Documentation
- ✓ API Testing
- ✓ Request/Response Examples
- ✓ Better Collaboration
- ✓ Reduced Development Time

byte^{XL}